Quiz System - Phase 1: Project Requirements Document

1. Problem Statement

This project involves building a console-based Java application for administering quizzes. The system should allow users to take predefined quizzes, answer multiple-choice questions, and receive feedback on their performance. Questions and quiz results are stored and managed via CSV files. This project will serve as a foundation for future enhancements like user management, web integration, and database migration.

2. Functional Requirements

1. The user starts the application and is prompted to enter their nickname.
2. The user sees a list of available quizzes and selects one.
3. Each quiz consists of a list of predefined question IDs, each referencing a question stored in a questions CSV file.
4. Each question displays the question text and a minimum of 3 answer options.
5. The user selects an answer by typing a number or letter corresponding to the option.
6. The system shuffles the options each time the quiz is taken.
7. After each quiz, the system calculates the score based on correct answers.
8. The user is shown a summary of their performance: total score, list of correct/incorrect answers, and what the right answers were.
9. The result is saved in a results CSV file with nickname, date, score, and quiz ID.
10. The scoreboard shows up to the top 10 scores for each quiz (sorted by score then date).

3. Non-Functional Requirements

1. All inputs from the user must be validated and sanitized.
2. Questions and results should persist using CSV files.
3. All files should support UTF-8 encoding.
4. Application must handle invalid or corrupt CSV data gracefully.
5. Use of Java best practices is mandatory (encapsulation, proper class design, method modularity).

4. Limitations

1. Only one user can take the quiz at a time.
2. Only multiple-choice questions are supported in this phase.
3. The quiz creator and question creator are stored as plain strings (nicknames) in CSVs.

4. User authentication is not implemented.
5. Questions cannot include media or formatting beyond plain text.

5. Acceptance Criteria

1. The application compiles and runs via CLI.
2. A user can start a quiz and complete it successfully.
3. Questions and results are read from and written to CSV files.
4. Option order is shuffled every time the quiz is taken.
5. Score and performance summary are accurate and correctly displayed.
6. The results file contains nickname, date, quiz ID, and score.
7. The scoreboard shows the top 10 results for the selected quiz.

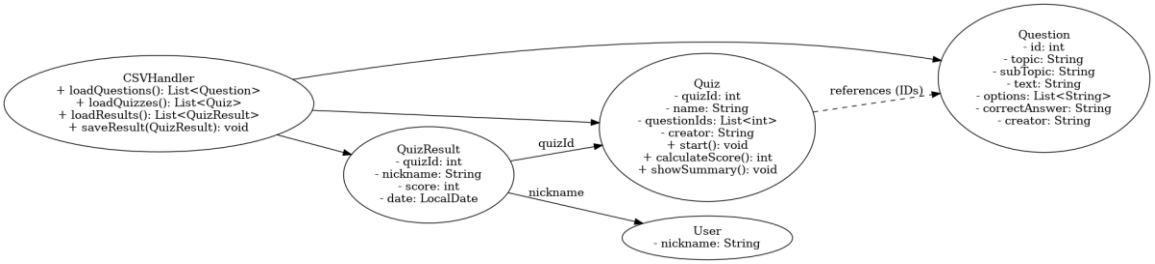
6. Milestones

1. Design and implement Question, Quiz, and User classes.
2. Implement functionality to load and parse CSV question data.
3. Implement quiz flow (question display, user input, validation, scoring).
4. Implement result saving and summary display.
5. Implement scoreboard logic (load, sort, top 10 display).
6. Final testing and bug fixing.
7. Document code and usage.

7. Diagrams

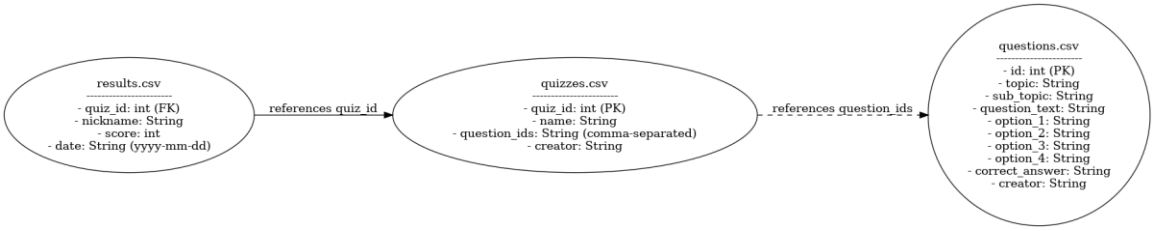
7.1 Class Diagram

The following diagram shows the classes used in Phase 1 and how they relate to each other:



7.2 Entity Relationship Diagram (ERD)

The following ERD represents the structure of CSV files used in Phase 1 and the relationships among them:



Note: The question IDs are stored as a comma-separated list in the 'quizzes.csv' file.

The results file must follow this column order: quiz_id, nickname, score, date (formatted as yyyy-mm-dd).

Note: The format and order of CSV columns should match the ERD specification for consistency.

- results.csv: Stores quiz attempt data including nickname, quiz ID, score, and date.
- quizzes.csv: Stores quiz metadata including quiz ID, name, list of question IDs, and creator.
- questions.csv: Stores individual questions including ID, text, options, correct answer, and creator.

6.1 File Naming and Structure Conventions